

Client Scheduling and Participation

Why scheduling matters

- Clients are heterogeneous and intermittent; naïve selection creates bias and slow rounds

Eligibility criteria (examples)

- Sufficient battery and charging state
- Unmetered/high-quality network; device idle and not in active use
- Minimum storage/CPU/GPU/RAM; recent app/version; security/attestation passed

Selection strategies

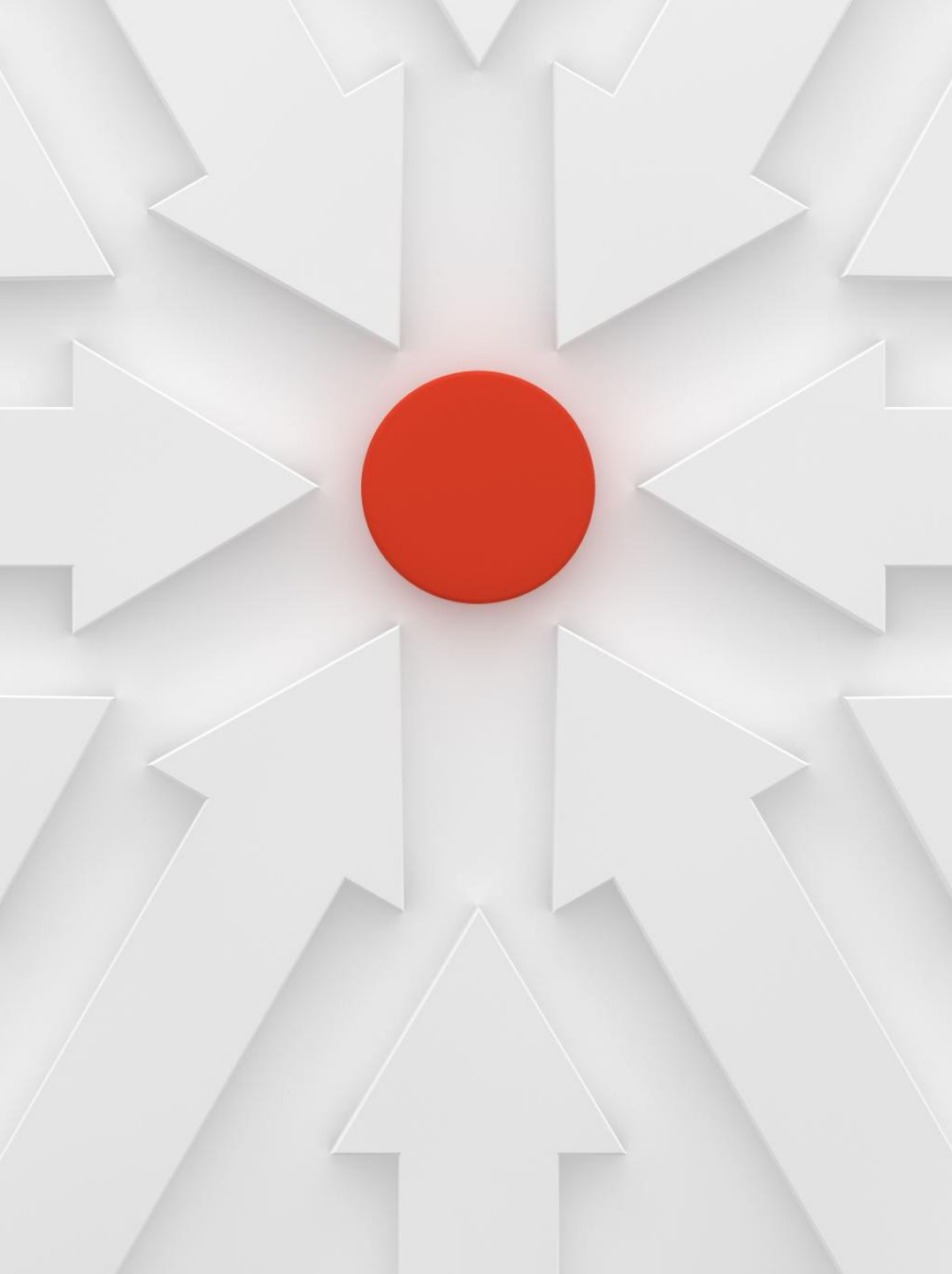
- Random sampling from eligible pool each round
- Stratified sampling by geography, device class, or data attributes to reduce bias
- Priority/credit systems to ensure fairness and fresh participation
- Cap per-client frequency to avoid overusing fast devices

Availability patterns to consider

- Time zones and diurnal patterns
- Usage behavior (work hours vs. overnight charging)
- Network variability by region/carrier; hardware capability differences

Robustness and performance

- Over-select (invite $K \gg \text{target}$) to tolerate dropouts; accept first N that check in
- Deadlines and partial aggregation; early stopping if enough updates arrive
- Adaptive cohort size: increase when stragglers are common
- Secure eligibility checks; anonymized telemetry for scheduling
- Metrics to monitor
- Round completion time, acceptance rate, dropout rate, coverage/fairness by strata
- Contribution quality/weight (examples seen, loss improvement)



Federated Learning Performance

Convergence factors

- Local steps vs. rounds; data and system heterogeneity; client sampling

Efficiency trade-offs

- More local steps → fewer rounds but risk of drift
- Communication cost vs. accuracy curve

Key metrics

- Rounds to target accuracy; bandwidth per round/total
- Participation rate and straggler impact
- On-device energy/time per round

Fault Tolerance and Reliability

Participation churn

- Over-invite; accept first N updates; retry/backoff; resumable uploads

Asynchronous/partial aggregation

- Quorum-based rounds; rolling aggregation; staleness-aware weighting



Staleness management

- Tag updates with model version; decay weights by lag; bound max staleness

Robustness to bad/malicious clients

- Norm clipping, validation-on-server, robust aggregators (median/trimmed mean)
- Outlier and consistency checks; reputation/blacklisting

Resilience engineering

- Idempotent jobs, checkpointing, rate limiting, circuit breakers
- Telemetry/alerts for dropout spikes and drift

A **quorum** is the minimum number of clients that must respond before the server proceeds with a training round. Instead of waiting for every single client, which could take forever, it aggregates updates once this representative group has reported in.

Performance Optimization

Smart client selection

- Score by data utility, device health, connectivity; diversify to avoid bias

Communication scheduling

- Prefer Wi-Fi + charging; predict availability windows; cache and batch updates

Adaptive training

- Per-device configs: batch size, local steps, precision; auto-tune online
- Early exit for slow/overheating devices

Bandwidth/compute efficiency

- Update compression: sparsify, quantize, delta + server-side momentum
- Partial model updates or adapters; server-side aggregation acceleration

Energy awareness

- Train only on charge or high battery; throttle CPU/GPU; budget per session

Metrics to track

- Time/round, completion rate, bytes/round, on-device energy, accuracy/round

